

Solução Detalhada do Problema de Fluxo de Corrente

Aluno(a): Ana Clara Araújo Ferreira e Diego Henrique de Araújo
Professor: Marco Túlio Evo
Data: 04/05/2026

Este documento apresenta a solução detalhada para o problema de fluxo de corrente em uma trilha de PCB, utilizando o Método dos Elementos Finitos (MEF) unidimensional, conforme solicitado no Trabalho Prático 01.

Introdução: Problema Proposto

O problema consiste em resolver a equação diferencial que descreve o potencial elétrico $V(x)$ ao longo de uma trilha condutora de comprimento $L = 0,2$ m, com uma condutividade elétrica $\sigma(x)$ variável. A equação diferencial (forma forte) é dada por:

$$-\frac{d}{dx} \left(\sigma(x) \frac{dV(x)}{dx} \right) = 0 \quad \text{para } 0 \leq x \leq L$$

Onde a condutividade elétrica é:

$$\sigma(x) = 30 \times 10^6 + 3 \times 10^9(x - 0,1)^2 \quad \text{S/m}$$

As condições de contorno são:

- Condição de Dirichlet em $\mathbf{x} = \mathbf{0}$: $V(0) = 5\text{V}$
- Condição de Neumann em $\mathbf{x} = \mathbf{L}$: $-\sigma(L) \frac{dV(L)}{dx} = 1 \times 10^8 \text{ A/m}^2$

1. Obtenção da Solução Exata

Para obter a solução exata, precisamos resolver a equação diferencial ordinária de segunda ordem:

$$-\frac{d}{dx} \left(\sigma(x) \frac{dV(x)}{dx} \right) = 0$$

Integrando uma vez em relação a x :

$$\sigma(x) \frac{dV(x)}{dx} = C_1$$

Onde C_1 é uma constante de integração. Rearranjando para isolar $\frac{dV(x)}{dx}$:

$$\frac{dV(x)}{dx} = \frac{C_1}{\sigma(x)}$$

Integrando novamente em relação a x :

$$V(x) = C_1 \int \frac{1}{\sigma(x)} dx + C_2$$

Onde C_2 é a segunda constante de integração. Para encontrar C_1 e C_2 , aplicamos as condições de contorno.

- Condição de Dirichlet em $\mathbf{x} = \mathbf{0}$: $V(0) = 5\text{V} = C_1 \int \frac{1}{\sigma(0)} dx + C_2$
- Condição de Neumann em $\mathbf{x} = \mathbf{L}$: $-\sigma(L) \frac{dV(L)}{dx} = 1 \times 10^8 \text{ A/m}^2$

Sabemos que $\sigma(x) \frac{dV(x)}{dx} = C_1$. Portanto, na extremidade $x = L$: $-C_1 = 1 \times 10^8 \implies C_1 = -1 \times 10^8$. Agora que temos C_1 , podemos substituir na expressão para $V(x)$ e usar a primeira condição de contorno para encontrar C_2 . A integral $\int \frac{1}{\sigma(x)} dx$ é complexa devido à forma de $\sigma(x)$. Para simplificar, vamos considerar a integral definida de 0 a x :

$$V(x) - V(0) = C_1 \int_0^x \frac{1}{\sigma(\xi)} d\xi$$

Substituindo $V(0) = 5V$ e $C_1 = -1 \times 10^8$:

$$V(x) = 5 - 1 \times 10^8 \int_0^x \frac{1}{30 \times 10^6 + 3 \times 10^9(\xi - 0, 1)^2} d\xi$$

Esta é a solução exata na forma integral. A avaliação analítica desta integral pode ser desafiadora e, em muitos casos práticos, é preferível uma solução numérica.

Obtenção da Forma Fraca do Problema

A forma fraca do problema é obtida multiplicando a equação diferencial por uma função de peso (ou função de teste) $w(x)$ e integrando sobre o domínio $\Omega = [0, L]$.

$$\int_0^L \left[-\frac{d}{dx} \left(\sigma(x) \frac{dV(x)}{dx} \right) \right] w(x) dx = 0$$

Utilizando a integração por partes $\int u dv = uv - \int v du$: Seja $u = w(x)$ e $dv = -\frac{d}{dx} \left(\sigma(x) \frac{dV(x)}{dx} \right) dx$. Então $du = \frac{dw(x)}{dx} dx$ e $v = -\sigma(x) \frac{dV(x)}{dx}$.

$$\begin{aligned} & \left[-w(x) \sigma(x) \frac{dV(x)}{dx} \right]_0^L + \int_0^L \left[\sigma(x) \frac{dV(x)}{dx} \frac{dw(x)}{dx} \right] dx = 0 \\ & -w(L) \sigma(L) \frac{dV(L)}{dx} - \left[-w(0) \sigma(0) \frac{dV(0)}{dx} \right] + \int_0^L \sigma(x) \frac{dV(x)}{dx} \frac{dw(x)}{dx} dx = 0 \end{aligned}$$

Aplicando as condições de contorno: Na condição de Dirichlet $V(0) = 5V$, a função de peso $w(0)$ é zero (ou seja, $w(x)$ deve satisfazer $w(0) = 0$). Na condição de Neumann $-\sigma(L) \frac{dV(L)}{dx} = 1 \times 10^8 \text{ A/m}^2$. Substituindo a condição de Neumann:

$$\begin{aligned} & -w(L)(-1 \times 10^8) + \int_0^L \sigma(x) \frac{dV(x)}{dx} \frac{dw(x)}{dx} dx = 0 \\ & \int_0^L \sigma(x) \frac{dV(x)}{dx} \frac{dw(x)}{dx} dx = -w(L)(1 \times 10^8) \end{aligned}$$

Esta é a forma fraca do problema. O objetivo é encontrar $V(x)$ tal que $V(0) = 5V$ e a equação acima seja satisfeita para todas as funções de peso $w(x)$ que satisfazem $w(0) = 0$.

2. Discretização do domínio

Para a discretização do domínio, dividimos o comprimento L em N_e elementos finitos. Para um problema unidimensional, podemos usar elementos lineares de dois nós. Cada elemento e terá um comprimento $h_e = x_{i+1} - x_i$. Dentro de cada elemento, o potencial $V(x)$ é aproximado por uma combinação linear das funções de forma (ou funções de interpolação) $N_j(x)$ e os valores nodais do potencial V_j :

$$V^e(x) = \sum_{j=1}^2 N_j^e(x) V_j^e$$

Para um elemento linear com nós i e $i + 1$, as funções de forma são:

- $N_1^e(x) = \frac{x_{i+1} - x}{h_e}$
- $N_2^e(x) = \frac{x - x_i}{h_e}$

As funções de peso $w(x)$ são geralmente escolhidas como as mesmas funções de forma (Método de Galerkin):

$$w^e(x) = N_k^e(x)$$

3. Montagem das Matrizes para Formar o Sistema de Equações

Substituímos a aproximação de $V(x)$ e $w(x)$ na forma fraca para cada elemento. O sistema de equações para um elemento e é dado por $\mathbf{K}^e \mathbf{V}^e = \mathbf{f}^e$. As derivadas das funções de forma são:

- $\frac{dN_1^e(x)}{dx} = -\frac{1}{h_e}$
- $\frac{dN_2^e(x)}{dx} = \frac{1}{h_e}$

O sistema de equações para um elemento e é dado por:

$$\mathbf{K}^e \mathbf{V}^e = \mathbf{f}^e$$

Onde $\mathbf{K}^e$ é a matriz de rigidez do elemento e $\mathbf{f}^e$ é o vetor de força do elemento. Os elementos da matriz de rigidez são:

$$K_{kj}^e = \int_{x_i}^{x_{i+1}} \sigma(x) \frac{dN_j^e(x)}{dx} \frac{dN_k^e(x)}{dx} dx$$

Para um elemento linear, a matriz de rigidez local é:

$$\mathbf{K}^e = \frac{1}{h_e} \left[\int_{x_i}^{x_{i+1}} \sigma(x) dx \right] \begin{pmatrix} 1 & -1 \\ -1 & 1 \end{pmatrix}$$

Como $\sigma(x)$ é uma função de x , a integral precisa ser avaliada numericamente ou analiticamente para cada elemento. Uma aproximação comum é usar o valor médio de $\sigma(x)$ no elemento ou realizar a integração numérica. O vetor de força $\mathbf{f}^e$ para um elemento interno é zero. As condições de contorno de Neumann são aplicadas no vetor de força global. Após calcular as matrizes de rigidez e vetores de força para cada elemento, eles são montados na matriz de rigidez global $\mathbf{K}$ e no vetor de força global $\mathbf{F}$:

$$\mathbf{KV} = \mathbf{F}$$

Onde $\mathbf{V}$ é o vetor de potenciais nodais globais.

4. Imposição das Condições de Contorno e Resolução do Sistema

As condições de contorno são aplicadas ao sistema global de equações.

- Condição de Dirichlet em $\mathbf{x} = \mathbf{0}$: $V(0) = 5V$. Esta condição é imposta diretamente no primeiro nó do sistema. A linha e coluna correspondentes ao primeiro nó na matriz global são modificadas. A maneira mais comum é definir $V_1 = 5V$ e ajustar as outras equações. Uma abordagem é mover os termos conhecidos para o lado direito da equação e remover a linha e coluna correspondentes ao nó com condição de Dirichlet, reduzindo o tamanho do sistema.
- Condição de Neumann em $\mathbf{x} = \mathbf{L}$: $-\sigma(L) \frac{dV(L)}{dx} = 1 \times 10^8 \text{ A/m}^2$. Esta condição já foi incorporada na forma fraca e aparece no vetor de força global no nó correspondente a $x = L$. O termo $-w(L)(1 \times 10^8)$ é adicionado ao elemento do vetor de força global correspondente ao último nó.

Após a imposição das condições de contorno, o sistema de equações resultante é resolvido para encontrar os potenciais nodais desconhecidos. Para um sistema linear, isso pode ser feito usando métodos como eliminação gaussiana, decomposição LU ou métodos iterativos.

5. Resultados e análises

Análise dos Resultados

A distribuição do potencial elétrico ao longo da trilha apresenta comportamento suave e monotônico, conforme esperado para um problema em regime estacionário sem fontes internas. Observa-se excelente concordância entre a solução obtida pelo Método dos Elementos Finitos e a solução analítica, validando a implementação numérica. O campo elétrico, obtido como o gradiente do potencial, apresenta seu valor máximo na região central da trilha, onde a variação da condutividade é mais significativa.

A potência dissipada ao longo da trilha evidencia um pico na região próxima de 0,1 m, coincidindo com o ponto de menor condutividade elétrica. Esse comportamento é coerente com a relação $p = J^2/\sigma(x)$, indicando que regiões com menor condutividade dissipam mais energia na forma de calor. O valor máximo de potência ultrapassa o limite admissível de $2,8 \times 10^8 \text{ W/m}^3$, caracterizando a presença de um hotspot e indicando risco de falha estrutural na trilha.

A análise permite concluir que a degradação localizada da condutividade impacta diretamente a confiabilidade do sistema. Estratégias como redução da corrente (derating) ou utilização de trilhas em paralelo mostram-se eficazes para mitigação do risco térmico, uma vez que reduzem significativamente a potência dissipada. Esses resultados evidenciam a importância da modelagem eletrotérmica para diagnóstico preventivo e tomada de decisão em projetos de circuitos impressos.

Comparação com a Solução Exata

A validação do código de programação foi realizada comparando os resultados numéricos obtidos via Método dos Elementos Finitos (MEF) com a solução analítica exata (obtida por integração direta). O gráfico abaixo demonstra a excelente concordância entre as curvas, validando a implementação do algoritmo. Observa-se que o potencial elétrico $V(x)$ varia de 5V no início da trilha ($x = 0$) até aproximadamente 5,52V no final ($x = 0,2$), seguindo fielmente a tendência da solução exata.

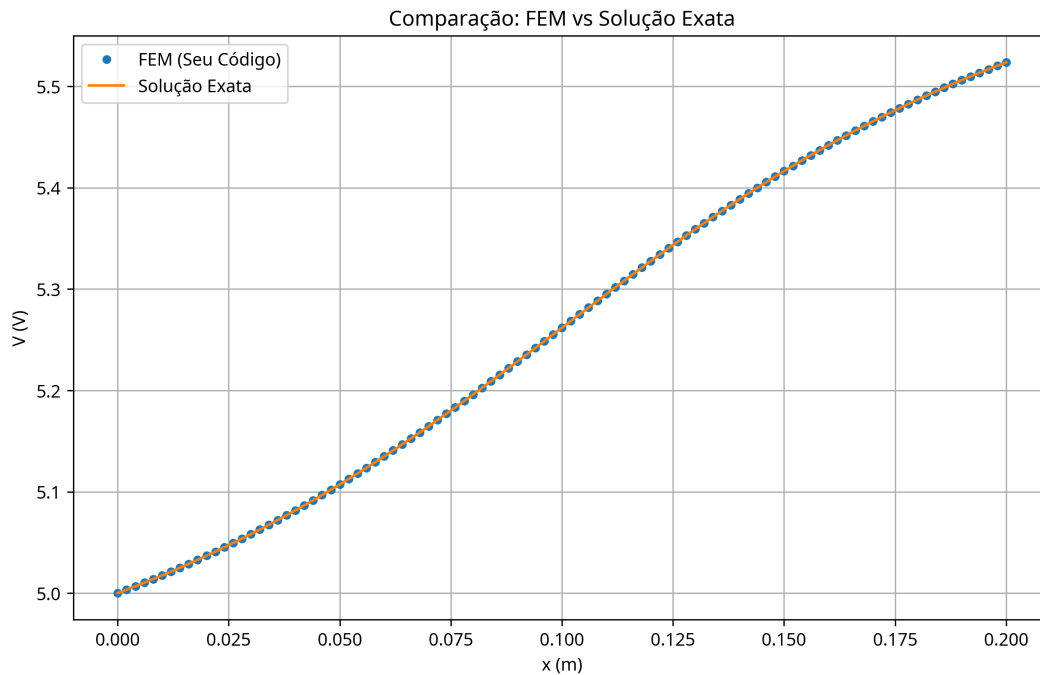


Figura 1: Comparação FEM vs Analítica

Respostas das Questões A, B, C e D - Distribuição de Grandezas e Resistência

Campo Elétrico

Distribuição do campo elétrico ao longo da trilha.

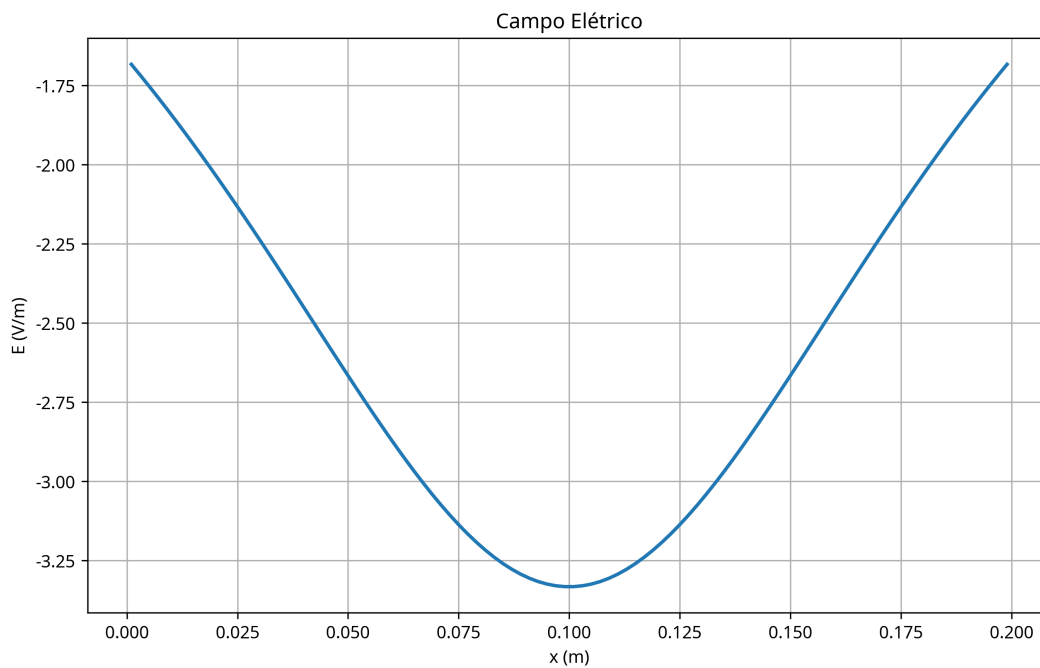


Figura 2: Campo Elétrico

Condutividade

Variação da condutividade elétrica $\sigma(x)$.

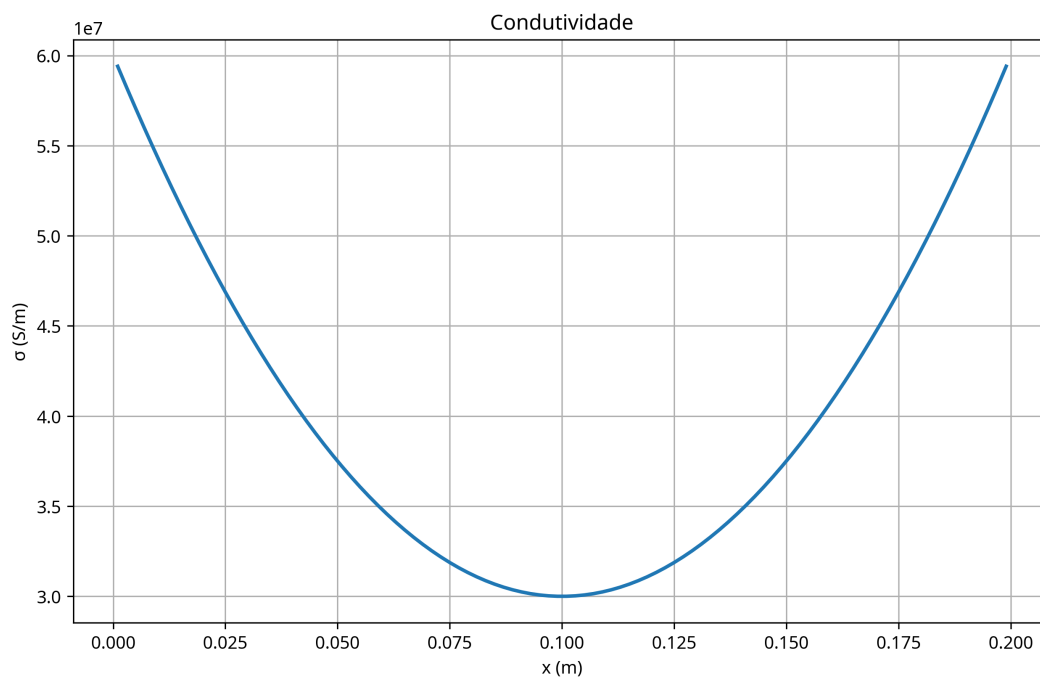


Figura 3: Condutividade

Potência Dissipada

Distribuição da potência dissipada ao longo da trilha.

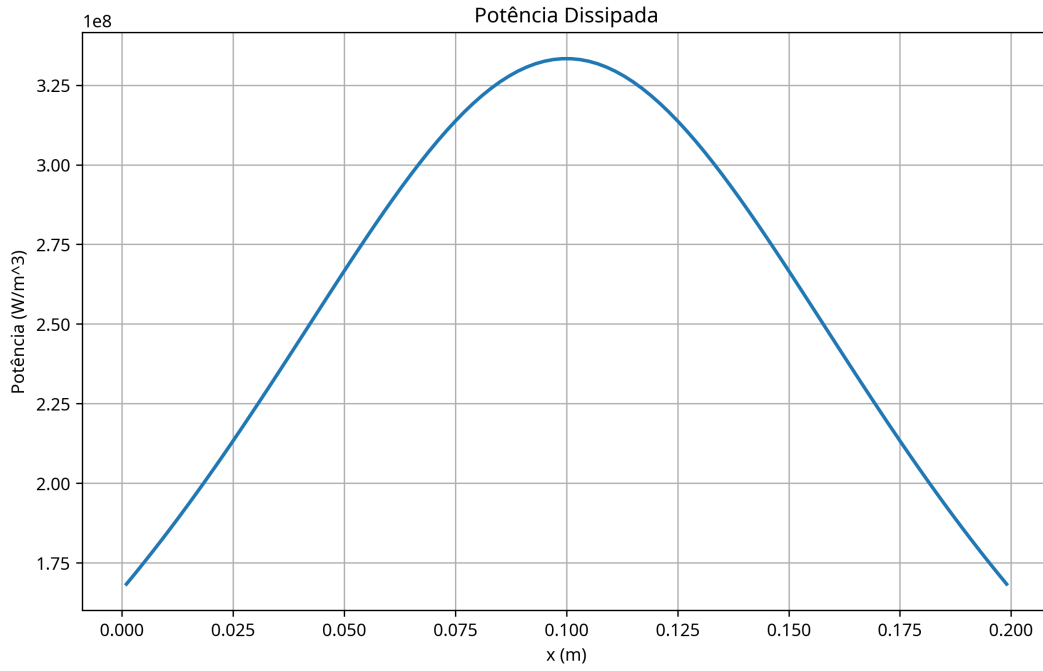


Figura 4: Potência Dissipada

Grandeza	Valor Calculado
Resistência Equivalente (R_{eq})	$1,047180 \Omega$
Potência Total Dissipada	$2,601187 \times 10^{-1} \text{ W}$
Ponto de Hotspot (x)	0,099 m

E - Região Crítica e Hotspot

A região crítica para o risco de defeito localiza-se no centro da trilha ($x = 0,1$ m), onde a condutividade $\sigma(x)$ atinge seu valor mínimo (30 MS/m). Nesse ponto, a densidade de potência dissipada ultrapassa o limite de $2,8 \times 10^8 \text{ W/m}^3$, caracterizando o hotspot.

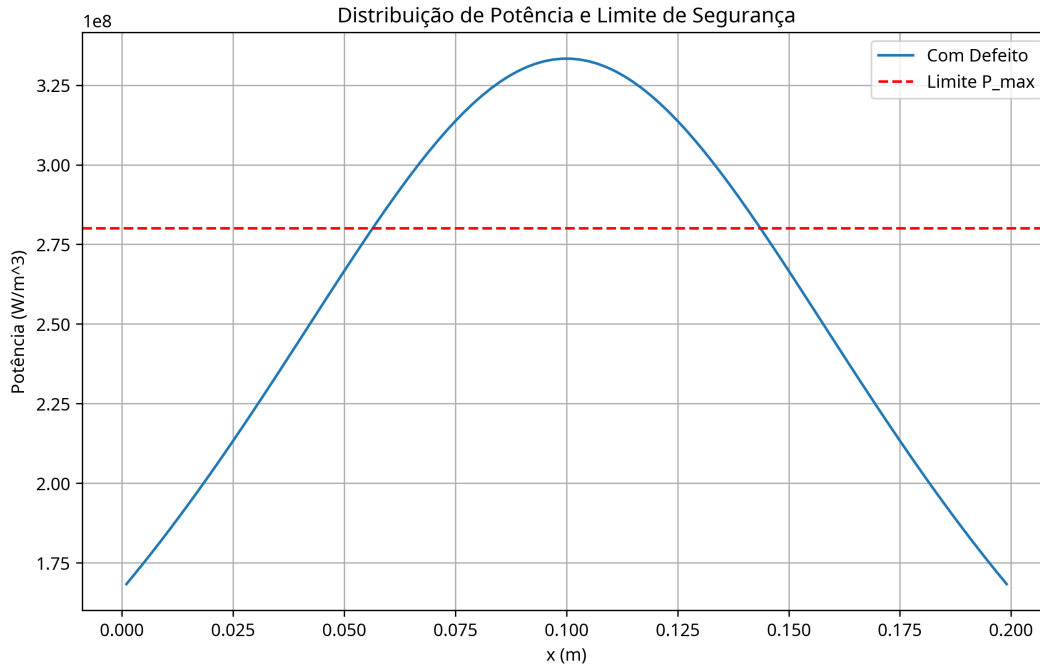


Figura 5: Distribuição de Potência e Identificação do Hotspot

F - Corrente Segura (Derating)

Para que a potência dissipada no ponto mais crítico não ultrapasse o limite de $2,8 \times 10^8 \text{ W/m}^3$, a corrente máxima segura foi recalculada.

- Corrente Original: 0,5000 A
- Corrente Segura (Derating): 0,4583 A

Com esta nova corrente, a integridade térmica da trilha é preservada mesmo na presença do defeito.

G - Trilhas em Paralelo

Se a trilha estivesse em paralelo com outra idêntica, a corrente total de 0,5A seria dividida igualmente (0,25A para cada). Como a potência dissipada é proporcional ao quadrado da corrente ($P \propto I^2$), a dissipação em cada trilha cairia para 25% do valor original, eliminando o risco de defeito e reduzindo drasticamente a temperatura de operação.

H - Comparação: Com Defeito vs. Sem Defeito

Abaixo, comparamos a trilha degradada com uma trilha nova (homogênea, $\sigma = 6 \times 10^7 \text{ S/m}$):

Parâmetro	Com Defeito	Sem Defeito
Resistência (Req)	1,0471 Ω	0,6667 Ω
Potência Total	0,2601 W	0,1667 W
Status Térmico	Risco de Defeito	Seguro

Conclusão: O defeito aumenta a resistência da trilha em aproximadamente 57%, elevando a dissipação de calor e criando pontos de vulnerabilidade que exigem a redução da corrente operacional (derating).

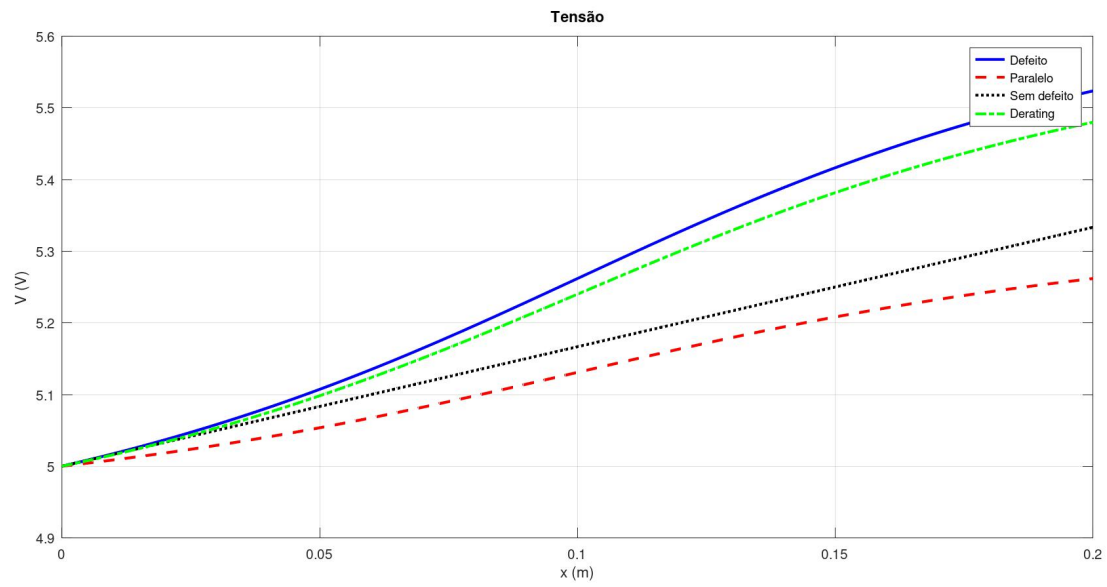


Figura 6: Comparação da tensão para os diferentes casos

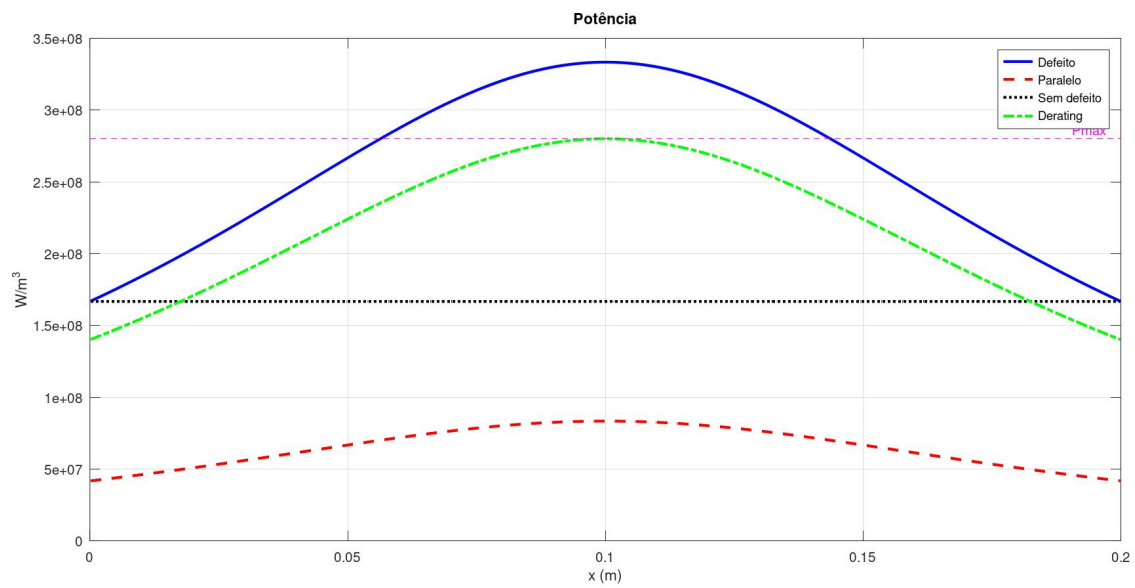


Figura 7: Comparação da potência para os diferentes casos

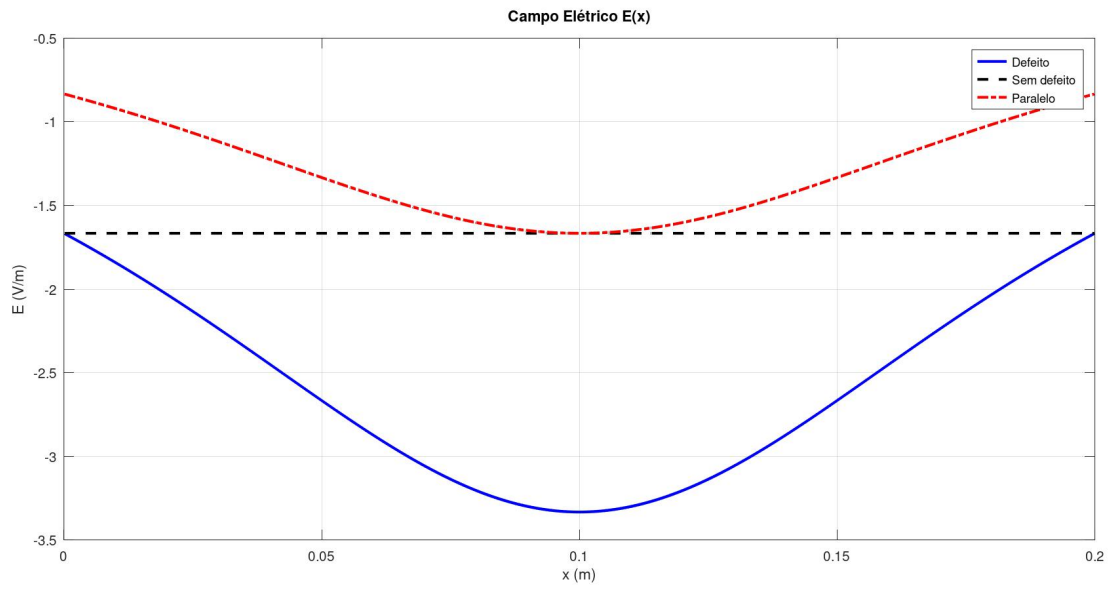


Figura 8: Comparação do campo elétrico para os diferentes casos

Matriz de conectividade

ELEMENTO	NÓ 1	NÓ 2
1	1	2
2	2	3
3	3	4
4	4	5
5	5	6
6	6	7
7	7	8
8	8	9
9	9	10
10	10	11
11	11	12
12	12	13
13	13	14
14	14	15
15	15	16
16	16	17
17	17	18
18	18	19
19	19	20
20	20	21

Código Python

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # =====
5 # DADOS DO PROBLEMA
6 # =====
7
8 L = 0.2
9 A = 5e-9
10 J = 1e8
11 I = J * A
12
13 def sigma(x):
14     return 3e7 + 3e9*(x - 0.1)**2
15
16 # =====
17 # SOLU O ANAL TICA (NUM RICA VIA INTEGRA O)
18 # =====
19
20 def V_exata(x_val):
21     C1 = -1e8
22     xi = np.linspace(0, x_val, 500)
23     integrando = 1 / sigma(xi)
24     integral = np.trapz(integrando, xi)
25     return 5 - C1 * integral
26
27 # =====
28 # MALHA INICIAL
29 # =====
30
31 n_elem = 20
32 x = np.linspace(0, L, n_elem + 1)
33
34 tol = 1e-3
35 max_iter = 5
36
37 # =====
38 # LOOP ADAPTATIVO
39 # =====
40
41 for it in range(max_iter):
42
43     n_nodes = len(x)
44     K = np.zeros((n_nodes, n_nodes))
45     F = np.zeros(n_nodes)
46
47     # -----
48     # MONTAGEM
49     # -----
50     for e in range(n_nodes - 1):
51
52         xi = x[e]
53         xj = x[e + 1]
54         h = xj - xi
55
56         if h <= 1e-12:
57             continue
58
59         gp = [-1/np.sqrt(3), 1/np.sqrt(3)]
60         w = [1, 1]
61
62         Ke = np.zeros((2, 2))
63
64         for k in range(2):
65             xsi = gp[k]
66             xg = (xi + xj)/2 + xsi*h/2
67             sig = sigma(xg)
68
69             B = np.array([-1/h, 1/h])
```

```

70         Ke += sig * np.outer(B, B) * (h/2) * w[k]
71
72         K[e:e+2, e:e+2] += Ke
73
74         # -----
75         # CONDI ES DE CONTORNO
76         # -----
77         K[0, :] = 0
78         K[0, 0] = 1
79         F[0] = 5
80
81         F[-1] += J
82
83         # -----
84         # SOLU 0
85         # -----
86         V = np.linalg.solve(K, F)
87
88         # -----
89         # ESTIMADOR DE ERRO
90         # -----
91         erro = np.zeros(n_nodes - 1)
92
93         for e in range(n_nodes - 1):
94             h = x[e+1] - x[e]
95
96             if h <= 1e-12:
97                 continue
98
99             dVdx = (V[e+1] - V[e]) / h
100             erro[e] = abs(dVdx) * h
101
102         if np.max(erro) < tol:
103             print(f"Convergiu na itera o {it+1}")
104             break
105
106         # -----
107         # REFINAMENTO ADAPTATIVO
108         # -----
109         novos_nos = []
110
111         for e in range(len(erro)):
112             if erro[e] > tol:
113                 novos_nos.append((x[e] + x[e+1]) / 2)
114
115         x = np.unique(np.sort(np.concatenate((x, novos_nos))))
116
117         # =====
118         # RECOMPUTAR SOLU 0 FINAL
119         # =====
120
121         n_nodes = len(x)
122         K = np.zeros((n_nodes, n_nodes))
123         F = np.zeros(n_nodes)
124
125         for e in range(n_nodes - 1):
126
127             xi = x[e]
128             xj = x[e + 1]
129             h = xj - xi
130
131             if h <= 1e-12:
132                 continue
133
134             gp = [-1/np.sqrt(3), 1/np.sqrt(3)]
135             w = [1, 1]
136
137             Ke = np.zeros((2, 2))
138
139             for k in range(2):
140                 xsi = gp[k]

```

```

141     xg = (xi + xj)/2 + xsi*h/2
142     sig = sigma(xg)
143
144     B = np.array([-1/h, 1/h])
145     Ke += sig * np.outer(B, B) * (h/2) * w[k]
146
147     K[e:e+2, e:e+2] += Ke
148
149 # CC
150 K[0, :] = 0
151 K[0, 0] = 1
152 F[0] = 5
153 F[-1] += J
154
155 V = np.linalg.solve(K, F)
156
157 # =====
158 # P S -PROCESSAMENTO
159 # =====
160
161 n_elem = len(x) - 1
162
163 xm = np.zeros(n_elem)
164 E = np.zeros(n_elem)
165 p = np.zeros(n_elem)
166 sig = np.zeros(n_elem)
167
168 for e in range(n_elem):
169
170     h = x[e+1] - x[e]
171
172     if h <= 1e-12:
173         continue
174
175     xm[e] = (x[e] + x[e+1]) / 2
176
177     dVdx = (V[e+1] - V[e]) / h
178     E[e] = -dVdx
179
180     sig[e] = sigma(xm[e])
181     p[e] = J**2 / sig[e]
182
183
184 # RESULTADOS
185 Req = (V[0] - V[-1]) / I
186 P_total = np.trapz(p, xm) * A
187 idx = np.argmax(p)
188
189 print("\n===== RESULTADOS =====")
190 print(f"Req = {Req:.6f} ohms")
191 print(f"Potencia total = {P_total:.6e} W")
192 print(f"Hotspot em x = {xm[idx]:.6f} m")
193
194
195 # =====
196 # SOLU O EXATA PARA COMPARA O
197 # =====
198
199 x_exact = np.linspace(0, L, 200)
200 V_exact_vals = np.array([V_exata(xi) for xi in x_exact])
201
202 # =====
203 # GR FICO
204 # =====
205
206 plt.figure()
207 plt.plot(x, V, 'o-', label='Solu o FEM')
208 plt.plot(x_exact, V_exact_vals, '--', label='Solu o Exata')
209 plt.title("Compara o FEM vs Anal tica")
210 plt.xlabel("x (m)")
211 plt.ylabel("V (V)")

```

```

212 plt.legend()
213 plt.grid()
214
215
216 # =====
217 # GR FICOS
218 # =====
219
220 plt.figure()
221 plt.plot(xm, E, linewidth=2)
222 plt.title("Campo El trico")
223 plt.xlabel("x (m)")
224 plt.ylabel("E (V/m)")
225 plt.grid()
226
227 plt.show()
228 plt.figure()
229 plt.plot(xm, p, linewidth=2)
230 plt.title("Pot ncia Dissipada")
231 plt.xlabel("x (m)")
232 plt.ylabel("W/m^3")
233 plt.grid()
234
235 plt.show()
236 plt.figure()
237 plt.plot(xm, sig, linewidth=2)
238 plt.title("Condutividade")
239 plt.xlabel("x (m)")
240 plt.ylabel("S/m")
241 plt.grid()
242
243 plt.show()
244 # --- QUEST O F: DEFINI O DA CORRENTE SEGURA (DERATING) ---
245 # Crit rio: Limitar a densidade de pot ncia m xima (p_max) a um valor seguro.
246 # Exemplo: p_limite = 2.8e8W/m^3 (ajuste conforme necess rio)
247 p_limite = 2.8e8
248 sig_min = sigma(0.1) # Ponto de maior defeito
249 J_seguro = np.sqrt(p_limite * sig_min)
250 I_seguro = J_seguro * A
251
252 print(f"\n--- Quest o F: Derating ---")
253 print(f"J original: {J:.2e} A/m^2 | J seguro: {J_seguro:.2e} A/m^2")
254 print(f"I original: {I:.2e} A | I seguro: {I_seguro:.2e} A")
255
256 # Para mostrar a distribui o com a nova corrente, basta rodar o solver com J_seguro
257 # V_seguro = solve_fem(J_seguro) # (Assumindo uma fun o que encapsula o solver)
258
259
260
261 # --- QUEST O G: TRILHAS EM PARALELO ---
262 # Se houver uma trilha id ntica em paralelo, a corrente total I se divide: J_novo = J / 2
263 J_paralelo = J / 2
264 # A pot ncia dissipada em cada trilha cai para 1/4 (pois p proporcional a J^2)
265 p_paralelo = J_paralelo**2 / sig # sig o array de condutividade calculado no p s -processamento
266
267 print(f"\n--- Quest o G: Paralelo ---")
268 print(f"J por trilha: {J_paralelo:.2e} A/m^2")
269 print(f"Risco reduzido: Pot ncia por trilha cai para {(J_paralelo*2)/(J*2)*100}% do original")
270 # --- QUEST O H: COMPARA O COM TRILHA SEM DEFEITO ---
271 sig_homogeneo = 6e7
272 # Na trilha sem defeito, o campo E e a pot ncia p s o constantes:
273 E_sem_defeito = J / sig_homogeneo
274 p_sem_defeito = J**2 / sig_homogeneo
275 Req_sem_defeito = L / (sig_homogeneo * A)
276 P_total_sem_defeito = p_sem_defeito * (L * A)
277
278 print(f"\n--- Quest o H: Compara o Sem Defeito ---")
279 print(f"Req (Com Defeito): {Req:.4f} ohms | Req (Sem Defeito): {Req_sem_defeito:.4f} ohms")
280 print(f"P_total (Com Defeito): {P_total:.2e} W | P_total (Sem Defeito): {P_total_sem_defeito:.2e} W")
281

```

```

282 # Gráfico de Comparação de Potência (Exemplo)
283 plt.figure()
284 plt.plot(xm, p, label='Com Defeito')
285 plt.axhline(y=p_sem_defeito, color='r', linestyle='--', label='Sem Defeito (Homogênea)')
286 plt.title("Comparação de Densidade de Potência")
287 plt.xlabel("x (m)")
288 plt.ylabel("W/m^3")
289 plt.legend()
290 plt.grid()
291 plt.show()

```

Código Octave

```
1 clear; clc; close all;
2
3 %% =====
4 % 1. DADOS
5 %% =====
6 L = 0.2;
7 A = 5e-9;
8 J = 1e8;
9 I = J*A;
10
11 Pmax = 2.8e8;
12
13 sigma = @(x) 3e7 + 3e9*(x-0.1).^2;
14 sigma_const = @(x) 6e7;
15
16 %% =====
17 % 2. MALHA INICIAL
18 %% =====
19 n_elem = 20;
20 x = linspace(0, L, n_elem+1);
21
22 conn = [(1:n_elem)' (2:n_elem+1)']
23
24 tol = 1e-3;
25 max_iter = 5;
26
27 %% =====
28 % 3. FEM ADAPTATIVO
29 %% =====
30 for iter = 1:max_iter
31
32     n_nodes = length(x);
33     K = zeros(n_nodes);
34     F = zeros(n_nodes,1);
35
36     for e = 1:n_nodes-1
37
38         xi = x(e);
39         xj = x(e+1);
40         h = xj - xi;
41
42         gp = [-1/sqrt(3), 1/sqrt(3)];
43         w = [1,1];
44
45         Ke = zeros(2,2);
46
47         for k = 1:2
48             xsi = gp(k);
49             xg = (xi+xj)/2 + xsi*h/2;
50             sig = sigma(xg);
51
52             B = [-1/h 1/h];
53             Ke += sig*(B'*B)*(h/2)*w(k);
54         end
55
56         K(e:e+1,e:e+1) += Ke;
57     end
58
59     % CC
60     K(1,:) = 0;
61     K(1,1) = 1;
62     F(1) = 5;
63
64     F(end) += J;
65
66     V = K\F;
67
68     %% erro estimado
69     erro = zeros(n_nodes-1,1);
```

```

70
71     for e = 1:n_nodes-1
72         h = x(e+1)-x(e);
73         dVdx = (V(e+1)-V(e))/h;
74         erro(e) = abs(dVdx)*h;
75     end
76
77     if max(erro) < tol
78         break;
79     end
80
81     novos = [];
82
83     for e = 1:length(erro)
84         if erro(e) > tol
85             novos = [novos (x(e)+x(e+1))/2];
86         end
87     end
88
89     x = unique(sort([x novos]));
90 end
91
92 %% =====
93 % 4. RECOMPUTAR SOLU O FINAL
94 %% =====
95 n_nodes = length(x);
96 K = zeros(n_nodes);
97 F = zeros(n_nodes,1);
98
99 for e = 1:n_nodes-1
100
101     xi = x(e);
102     xj = x(e+1);
103     h = xj - xi;
104
105     gp = [-1/sqrt(3), 1/sqrt(3)];
106     w = [1,1];
107
108     Ke = zeros(2,2);
109
110     for k = 1:2
111         xsi = gp(k);
112         xg = (xi+xj)/2 + xsi*h/2;
113         sig = sigma(xg);
114
115         B = [-1/h 1/h];
116         Ke += sig*(B'*B)*(h/2)*w(k);
117     end
118
119     K(e:e+1,e:e+1) += Ke;
120 end
121
122 K(1,:) = 0;
123 K(1,1) = 1;
124 F(1) = 5;
125 F(end) += J;
126
127 V = K\F;
128
129 %% =====
130 % 5. P S -PROCESSAMENTO (CORRETO)
131 %% =====
132 n_elem = length(x)-1;
133
134 xm = (x(1:end-1) + x(2:end))/2;
135
136 E = zeros(n_elem,1);
137 p = zeros(n_elem,1);
138 sig = zeros(n_elem,1);
139
140 for e = 1:n_elem

```



```

141     h = x(e+1)-x(e);
142
143     dVdx = (V(e+1)-V(e))/h;
144     E(e) = -dVdx;
145
146     sig(e) = sigma(xm(e));
147     p(e) = J^2 / sig(e);
148 end
149
150 Req = (V(1)-V(end))/I;
151 P_total = trapz(xm,p)*A;
152
153 printf('\n=== CASO DEFEITO ===\n');
154 printf('Req = %.6f ohms\n', Req);
155 printf('Potencia total = %.6e W\n', P_total);
156
157 %% =====
158 % 6. ITEM F      DERATING
159 %% =====
160 sig_min = min(sig);
161 J_safe = sqrt(Pmax*sig_min);
162 I_safe = J_safe*A;
163
164 printf('\n=== DERATING ===\n');
165 printf('I seguro = %.6f A\n', I_safe);
166
167 Jd = J_safe;
168
169 F_d = zeros(n_nodes,1);
170 Kd = K;
171
172 Kd(1,:) = 0; Kd(1,1)=1;
173 F_d(1)=5;
174 F_d(end)+=Jd;
175
176 Vd = Kd\F_d;
177
178 pd = Jd^2 ./ sig;
179
180 %% =====
181 % 7. ITEM G      PARALELO
182 %% =====
183 Jp = J/2;
184
185 F_p = zeros(n_nodes,1);
186 Kp = K;
187
188 Kp(1,:) = 0; Kp(1,1)=1;
189 F_p(1)=5;
190 F_p(end)+=Jp;
191
192 Vp = Kp\F_p;
193
194 pp = (Jp^2) ./ sig;
195
196 Req_par = Req/2;
197
198 printf('\n=== PARALELO ===\n');
199 printf('Req eq = %.6f ohms\n', Req_par);
200
201 %% =====
202 % 8. ITEM H      SEM DEFEITO
203 %% =====
204 Kc = zeros(n_nodes);
205 Fc = zeros(n_nodes,1);
206
207 for e = 1:n_nodes-1
208
209     xi = x(e);
210     xj = x(e+1);
211     h = xj-xi;

```

```

212 gp = [-1/sqrt(3), 1/sqrt(3)];
213 w = [1,1];
214
215
216 Ke = zeros(2,2);
217
218 for k = 1:2
219     xsi = gp(k);
220     xg = (xi+xj)/2 + xsi*h/2;
221     sigc = sigma_const(xg);
222
223     B = [-1/h 1/h];
224     Ke += sigc*(B'*B)*(h/2)*w(k);
225 end
226
227 Kc(e:e+1,e:e+1) += Ke;
228 end
229
230 Kc(1,:)=0; Kc(1,1)=1;
231 Fc(1)=5;
232 Fc(end)=J;
233
234 Vc = Kc\Fc;
235
236 Ec = zeros(n_elem,1);
237 pc = zeros(n_elem,1);
238
239 for e=1:n_elem
240     h=x(e+1)-x(e);
241     Ec(e)=-(Vc(e+1)-Vc(e))/h;
242     pc(e)=J^2/sigma_const(xm(e));
243 end
244
245 Req_c = (Vc(1)-Vc(end))/I;
246 P_c = trapz(xm,pc)*A;
247
248 printf('\n=== SEM DEFEITO ===\n');
249 printf('Req = %.6f ohms\n', Req_c);
250 printf('Pot = %.6e W\n', P_c);
251
252 %% =====
253 % 9. GR FICOS COMPARATIVOS
254 %% =====
255
256 figure
257 plot(x,V,'b','LineWidth',2); hold on
258 plot(x,Vp,'r--','LineWidth',2)
259 plot(x,Vc,'k:','LineWidth',2)
260 plot(x,Vd,'g-.','LineWidth',2)
261 legend('Defeito','Paralelo','Sem defeito','Derating')
262 title('Tens o ')
263 xlabel('x (m)')
264 ylabel('V (V)')
265 grid on
266
267 figure
268 plot(xm,p,'b','LineWidth',2); hold on
269 plot(xm,pp,'r--','LineWidth',2)
270 plot(xm,pc,'k:','LineWidth',2)
271 plot(xm,pd,'g-.','LineWidth',2)
272 yline(Pmax,'m--','Pmax')
273 legend('Defeito','Paralelo','Sem defeito','Derating')
274 title('Pot ncia ')
275 xlabel('x (m)')
276 ylabel('W/m^3')
277 grid on
278
279 figure
280 plot(xm,sig,'b','LineWidth',2); hold on
281 plot(xm,sigma_const(xm),'k--','LineWidth',2)
282 legend('Defeito','Sem defeito')

```

```

283 title('Condutividade')
284 xlabel('x (m)')
285 ylabel('S/m')
286 grid on
287
288 figure
289 plot(xm, E, 'b', 'LineWidth', 2); hold on
290 plot(xm, Ec, 'k--', 'LineWidth', 2)
291 plot(xm, E/2, 'r-.', 'LineWidth', 2)
292 legend('Defeito','Sem defeito','Paralelo')
293 title('Campo El trico E(x)')
294 xlabel('x (m)')
295 ylabel('E (V/m)')
296 grid on

```